

Compte Rendu du TP MLOps

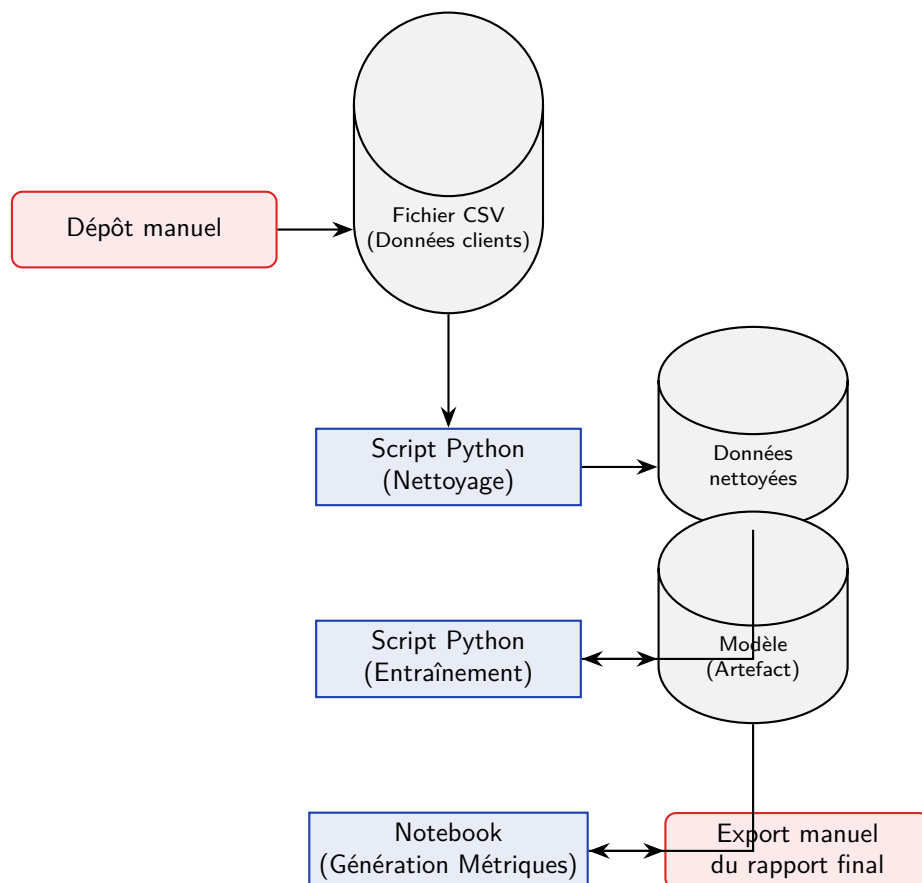
Partie II : Structuration d'un pipeline data/ML simple

Module : MLOps & DataOps

Réalisé par :
Ayman Chabbaki
Groupe 1

Exercice 1 — Cartographie du pipeline existant

1. Schéma du pipeline actuel



2. Liste des points de fragilité

- **Interventions manuelles :** Le dépôt initial et l'export final dépendent d'une action humaine, augmentant le risque d'oubli ou d'erreur.
- **Absence de versionnement des données :** Si le fichier CSV est écrasé, il est impossible de reproduire les anciens résultats.
- **Dépendance aux Notebooks pour l'évaluation :** Difficiles à automatiser et à versionner proprement avec Git.
- **Rupture de flux (Pas d'orchestrateur) :** Les scripts doivent être lancés séquentiellement à

la main. Pas de gestion d'erreur automatique.

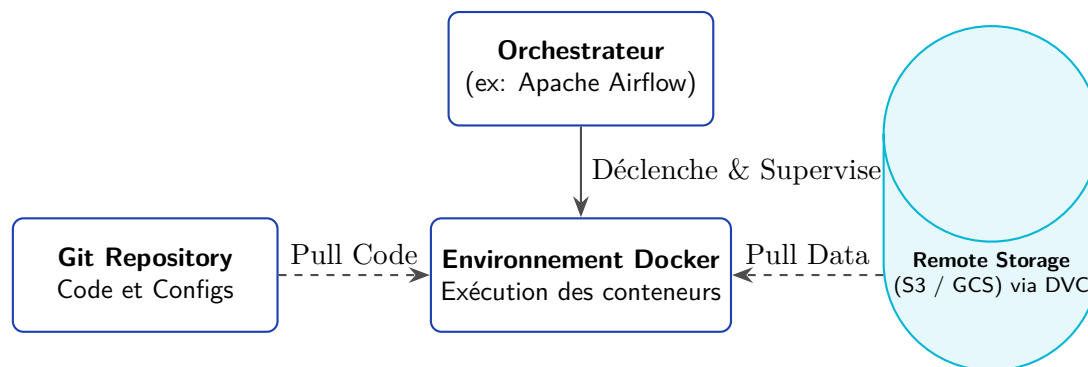
- **Absence de registre de modèles** : L'artefact n'est pas tracé.

Exercice 2 — Identification des éléments versionnables

Élément	Pourquoi versionner ?	Outil / Mécanisme
Code source (Scripts)	Suivre l'évolution logique du code, permettre la collaboration et le retour en arrière.	Git
Jeux de données	Garantir la reproductibilité absolue.	DVC
Configurations	Tracer les hyperparamètres impactant la performance.	Git
Artefacts (Modèles)	Archiver les modèles pour pouvoir les déployer ou les comparer.	MLflow / DVC
Env. d'exécution	Assurer que le code s'exécute de la même manière partout.	Docker

Exercice 3 — Proposition d'architecture simple

1. Schéma d'architecture



2. Explication de l'architecture

Dans cette architecture cible, **Git** centralise l'ensemble de la logique métier (scripts) ainsi que les fichiers de configuration et les définitions d'environnement (Dockerfile). La donnée (fichiers CSV, modèles) n'est pas stockée dans Git, mais gérée par **DVC**, qui utilise un stockage distant tout en conservant un pointeur léger versionné dans Git. **L'orchestrateur** a pour rôle de définir le graphe d'exécution (DAG), de planifier les tâches, de gérer les dépendances entre elles et d'assurer la gestion des erreurs. La **reproductibilité** est garantie par l'exécution de chaque étape au sein d'un conteneur **Docker** isolé, figeant les dépendances. Le **déclenchement** du pipeline peut être automatisé via l'orchestrateur, de manière planifiée ou événementielle.

Exercice 4 — Lecture critique d'un pipeline artisanal

La transition d'un script exploratoire vers un produit logiciel robuste nécessite une analyse critique. Le pipeline actuel présente de multiples failles structurelles :

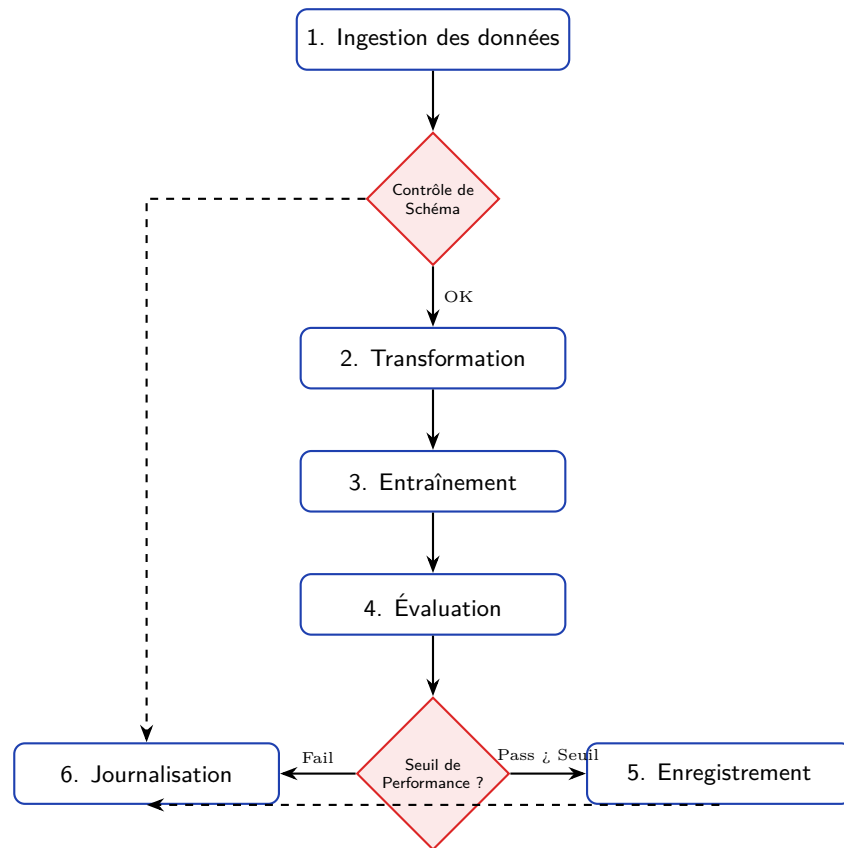
Dépendance aux manipulations manuelles et faible reproductibilité : Le dépôt manuel et l'export des résultats sont des goulots d'étranglement. L'intervention humaine introduit des risques d'erreurs. Sans environnement virtuel strict (comme Docker), le projet souffre du syndrome du "ça marche sur ma machine".

Manque de modularité et difficulté de passage à l'échelle : L'utilisation de scripts monolithiques ou de Notebooks pour la génération de métriques empêche la réutilisation du code. Si le volume de données augmente, l'architecture nécessitera de charger tout le CSV en mémoire, rendant le calcul distribué impossible.

Absence de versionnement et de contrôle automatisé : L'absence de traçabilité signifie que l'équipe navigue à vue. Si le modèle se dégrade, il est impossible d'identifier l'origine. Aucun test automatisé n'est présent, une anomalie dans le CSV d'entrée se propagera silencieusement.

Exercice 5 — Organisation d'un workflow cible

1. Diagramme de workflow structuré



2. Justification des choix de structuration

Ce workflow cible transforme l'approche chaotique en un pipeline robuste :

- **Ordre et Dépendances** : L'exécution est strictement séquentielle et conditionnelle.
- **Points de contrôle (Gates)** : Le *Contrôle de Schéma* s'assure de la validité de la donnée avant traitement. Le *Seuil de performance* garantit que seul un modèle performant est enregistré.
- **Sorties** : Chaque étape génère une sortie intermédiaire traçable (données validées, features, modèle, métriques).
- **Journalisation transverse** : Toutes les étapes émettent des logs pour déboguer rapidement une défaillance.
- **Déclenchement** : Le pipeline peut être déclenché quotidiennement (batch inference) ou lors de l'arrivée d'un nouveau fichier (événementiel).